CO2 – Assignment 2

NAME:HARINI.S(192412098)

Subject: C Programming
Assignment: CO2 – Assignment 2
Topic: Precedence, Associativity, Short-Circuit Evaluation, Bitwise Operators, Decision-Making and Loops

QUESTION 1 – PRECEDENCE, ASSOCIATIVITY AND THE DISCOUNT BUG

1.1 Problem explanation

A Jan Aushadhi government pharmacy calculates the amount payable by customers.

The billing rules are:

- Calculate the total price using unit price multiplied by quantity.
- Give a 15% discount to senior citizens.
- Add 5% GST.
- Provide an additional Rs. 50 discount when the total amount is greater than Rs. 1000.

The developer wrote the complete calculation in a single expression. This can cause calculation errors because of operator precedence, associativity and integer division.

1.2 Original C code

```
None
total = price * qty - price * qty * 15 / 100 * senior
        + price * qty * 5 / 100;

net = total > 1000 ? total - 50 : total;

count = count++ + 1;
```

1.3 Explanation of operators

Operator	Meaning	Precedence
*	Multiplication	High
/	Division	High
+	Addition	Lower
-	Subtraction	Lower
>	Greater than	Relational
?:	Conditional operator	Lower
++	Increment	High

Multiplication and division are evaluated before addition and subtraction. Operators at the same precedence level generally associate from left to right.

1.4 Corrected calculation

The calculation should be divided into smaller steps:

1. Calculate gross amount.
2. Calculate senior-citizen discount.
3. Subtract the discount.
4. Calculate GST on the discounted amount.
5. Calculate the final amount.
6. Apply the additional Rs. 50 discount if applicable.

1.5 Complete C program

```
None

#include <stdio.h>

int main()
{
    double price, gross, discount, taxable;
    double gst, total, net;
    int qty, senior;

    printf("Enter medicine price: ");
    scanf("%lf", &price);

    printf("Enter quantity: ");
```

```
scanf("%d", &qty);

printf("Senior citizen? (1-Yes, 0-No): ");
scanf("%d", &senior);

if (price < 0 || qty <= 0 ||
    (senior != 0 && senior != 1))
{
    printf("Invalid input");
    return 1;
}

gross = price * qty;

if (senior == 1)
    discount = gross * 0.15;
else
    discount = 0;

taxable = gross - discount;

gst = taxable * 0.05;

total = taxable + gst;

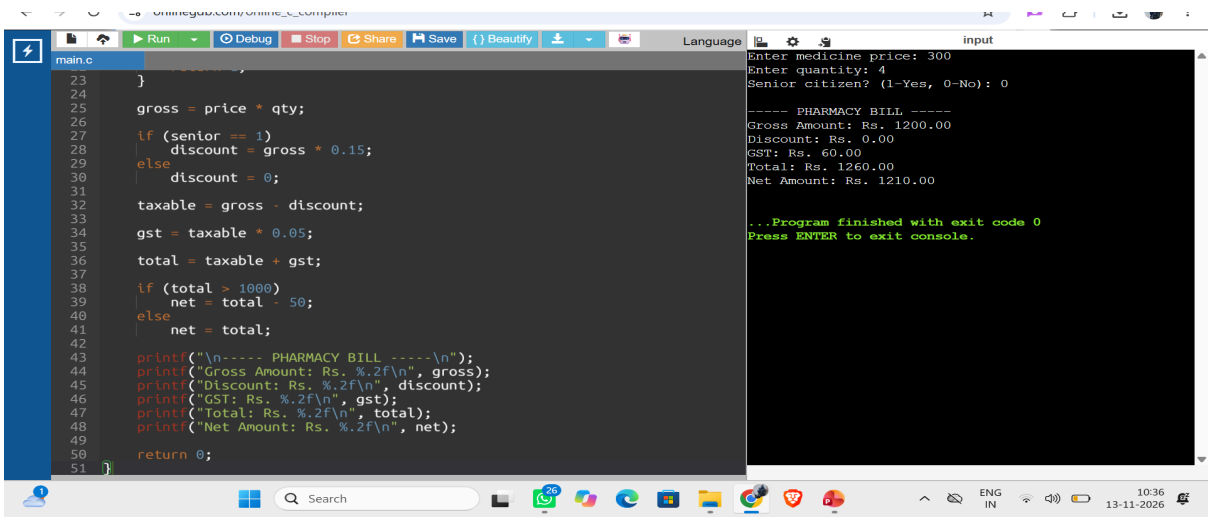
if (total > 1000)
```

```
        net = total - 50;
    else
        net = total;

    printf("\n----- PHARMACY BILL -----\n");
    printf("Gross Amount: Rs. %.2f\n", gross);
    printf("Discount: Rs. %.2f\n", discount);
    printf("GST: Rs. %.2f\n", gst);
    printf("Total: Rs. %.2f\n", total);
    printf("Net Amount: Rs. %.2f\n", net);

    return 0;
}
```

1.6 output



1.7 Three bill predictions

Bill	Price	Quantity	Senior	Gross
A	100	2	Yes	200
B	300	4	No	1200
C	250	5	Yes	1250

Using the original expression:

- Bill A: Total = Rs. 180; Net = Rs. 180.
- Bill B: Total = Rs. 1260; Net = Rs. 1210.
- Bill C: Total = Rs. 1125; Net = Rs. 1075.

These values follow the original integer-arithmetic expression. The corrected program uses floating-point calculations and applies GST after the senior discount.

1.8 Undefined behaviour

The statement:

```
None
count = count++ + 1;
```

is undefined behaviour in C because it modifies and accesses `count` without a valid sequencing relationship.

The correct way to increment a variable is:

```
None
count++;
```

The compiler may issue a warning with:

```
None
gcc program.c -Wall -Wextra
```

1.9 Time and space complexity

- Time complexity: O(1).
- Space complexity: O(1).

The number of calculations remains fixed regardless of the quantity of medicine.

1.10 Conclusion

- Operator precedence determines the order of calculations.
- Integer division can remove fractional values.
- Undefined behaviour must be avoided.
- Intermediate variables improve readability and debugging.
- The corrected program calculates discounts, GST and the final amount separately.

QUESTION 2 – WHEN DOES SHORT-CIRCUIT EVALUATION MATTER?

2.1 Problem explanation

A railway ticket counter provides concessions to passengers under two conditions:

1. The passenger is a senior citizen AND the ticket class is eligible.
2. The passenger has a valid disability card.

The disability-card verification function connects to a server and takes additional processing time.

The objective is to understand how logical operators reduce unnecessary function calls.

2.2 Logical operators

Operator	Meaning	Behaviour
`&&`	Logical AND	Stops when the first condition is false
`||`	Logical OR	Stops when the first condition is true
`&`	Bitwise AND	Evaluates both operands
`|`	Bitwise OR	Evaluates both operands

2.3 Short-circuit evaluation

Consider:

```
None
if (senior && eligible)
```

If `senior` is false, C does not evaluate `eligible`.

Similarly:

```
None
if (senior || verify_card())
```

If `senior` is true, the verification function is not called.

This reduces unnecessary processing.

2.4 Complete C program

None

```
#include <stdio.h>

int calls = 0;

int verify_card(int id)
{
    calls++;

    printf("Verifying card for ID %d\n", id);

    if (id == 7)
        return 1;

    return 0;
}

int main()
{
    int senior[5] = {1, 1, 0, 0, 1};
    int eligible[5] = {1, 0, 1, 0, 1};
    int id[5] = {2, 7, 7, 3, 4};

    int i, qualifies;

    for (i = 0; i < 5; i++)
```

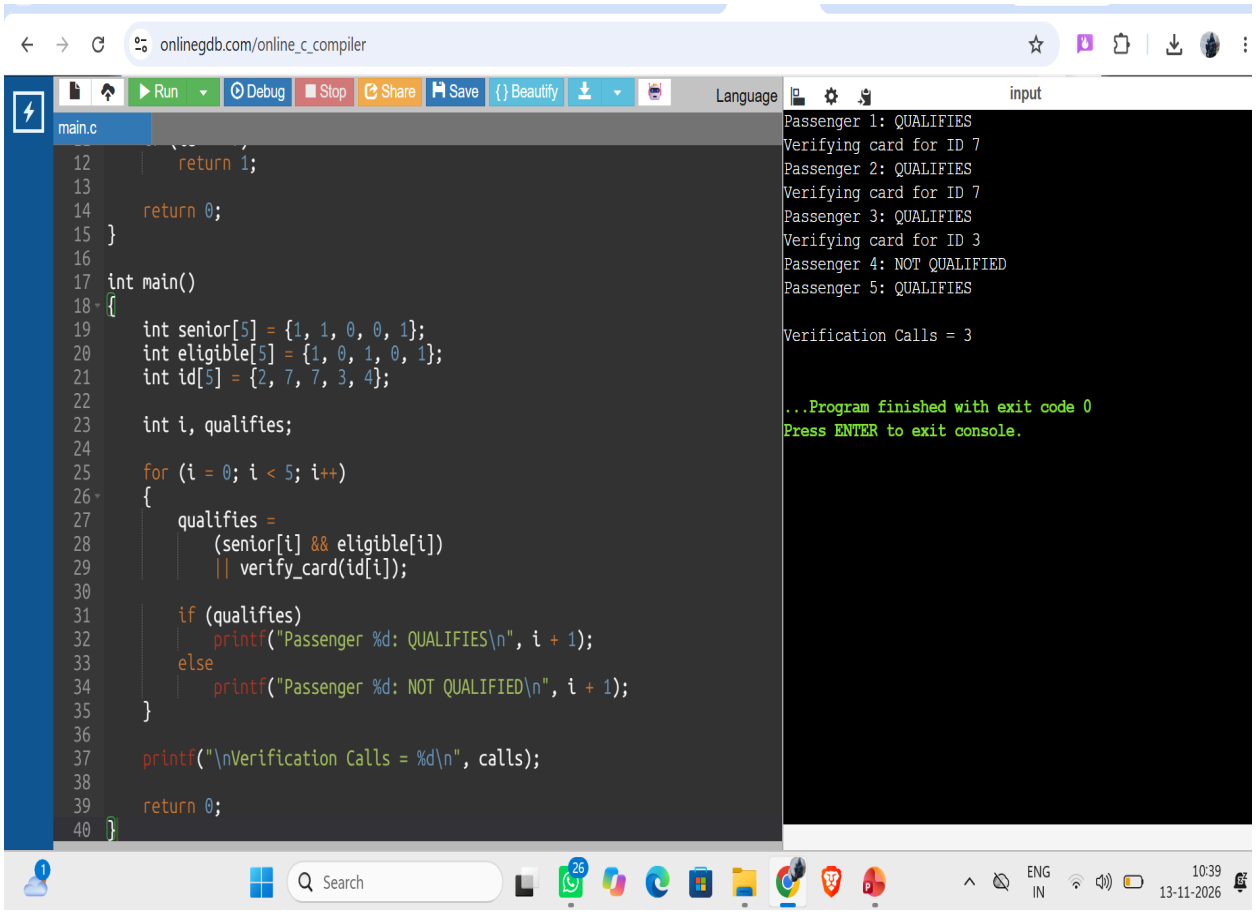
```
    {
        qualifies =
            (senior[i] && eligible[i])
            || verify_card(id[i]);

        if (qualifies)
            printf("Passenger %d: QUALIFIES\n", i + 1);
        else
            printf("Passenger %d: NOT QUALIFIED\n", i + 1);
    }

    printf("\nVerification Calls = %d\n", calls);

    return 0;
}
```

2.5 output



2.6 Explanation

There are five passengers.

- Passenger 1 satisfies both senior and eligible conditions. Server verification is skipped.
- Passenger 2 does not satisfy both conditions. Server verification is performed.
- Passenger 3 is not a senior citizen. Server verification is performed.
- Passenger 4 does not satisfy the first condition. Server verification is performed.
- Passenger 5 satisfies both conditions. Server verification is skipped.

Therefore, the total number of verification calls is 3.

2.7 Comparison of condition ordering

For 1,000 passengers, assume 300 passengers satisfy both senior and eligible conditions.

| Method | Verification calls |

|---|---:|

| `(senior && eligible) || verify_card()` | 700 |

| `verify_card() || (senior && eligible)` | 1000 |

| Mandatory verification for audit | 1000 |

The first condition is more efficient when it frequently succeeds and is inexpensive to evaluate.

2.8 Changed audit requirement

If the verification function must record every passenger in an audit log, the function must be called separately.

```
None
int card_valid = verify_card(id);

int qualifies =
    (senior && eligible) || card_valid;
```

This ensures that every passenger is verified and audited.

2.9 Time and space complexity

- Time complexity: O(n) for n passengers.
- Space complexity: O(1) additional space when processing one passenger at a time.

2.10 Conclusion

- Short-circuit evaluation avoids unnecessary function calls.
- `&&` and `||` improve efficiency.
- `&` and `|` do not provide short-circuit behaviour.
- Condition ordering affects processing time.

- Mandatory audit requirements must not be bypassed through short-circuit evaluation.

QUESTION 3 – ENGINEER A DEVICE-STATUS REGISTER WITH BITWISE OPERATORS

3.1 Problem explanation

A smart-home controller must store the ON/OFF status of up to eight electrical devices using a single `unsigned char`.

Each bit represents one device.

For example:

Bit position	Device	Status
Bit 0	Light	ON/OFF
Bit 1	Fan	ON/OFF
Bit 2	AC	ON/OFF
Bit 3	Heater	ON/OFF
Bit 4	Pump	ON/OFF
Bit 5	Device 6	ON/OFF

Bit 6	Device 7	ON/OFF
Bit 7	Device 8	ON/OFF

A value of 1 indicates ON, while 0 indicates OFF.

3.2 Bitwise operators

Operator	Operation	Purpose
&	AND	Check or clear bits
|	OR	Set bits
^	XOR	Toggle bits
~	NOT	Reverse bits
<<	Left shift	Create bit masks
>>	Right shift	Read bit positions

3.3 Algorithm

1. Initialize the register to zero.
2. Use left shift to create a device mask.
3. Use OR to switch a device ON.
4. Use AND with complement to switch a device OFF.
5. Use XOR to toggle a device.
6. Use AND to check the device status.
7. Display the register in binary.
8. Count the number of ON devices.

3.4 Complete C program – TODO 1 to TODO 4

None

```
#include <stdio.h>

enum {
    LIGHT = 0,
    FAN = 1,
    AC = 2,
    HEATER = 3,
    PUMP = 4
};

unsigned char switch_on(unsigned char reg, int dev)
{
    return (unsigned char)(reg | (1u << dev));
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return (unsigned char)(reg & ~(1u << dev));
}

unsigned char toggle(unsigned char reg, int dev)
{
    return (unsigned char)(reg ^ (1u << dev));
}
```

```
int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;
}

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--)
    {
        putchar(((reg >> bit) & 1u) ? '1' : '0');
    }

    putchar('\n');
}

int count_on(unsigned char reg)
{
    int count = 0;

    for (int bit = 0; bit < 8; bit++)
    {
        if (reg & (1u << bit))
            count++;
    }

    return count;
```

```
}

int main()
{
    unsigned char reg = 0;

    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);

    printf("Register after switching ON:\n");
    show(reg);

    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);

    printf("Register after operations:\n");
    show(reg);

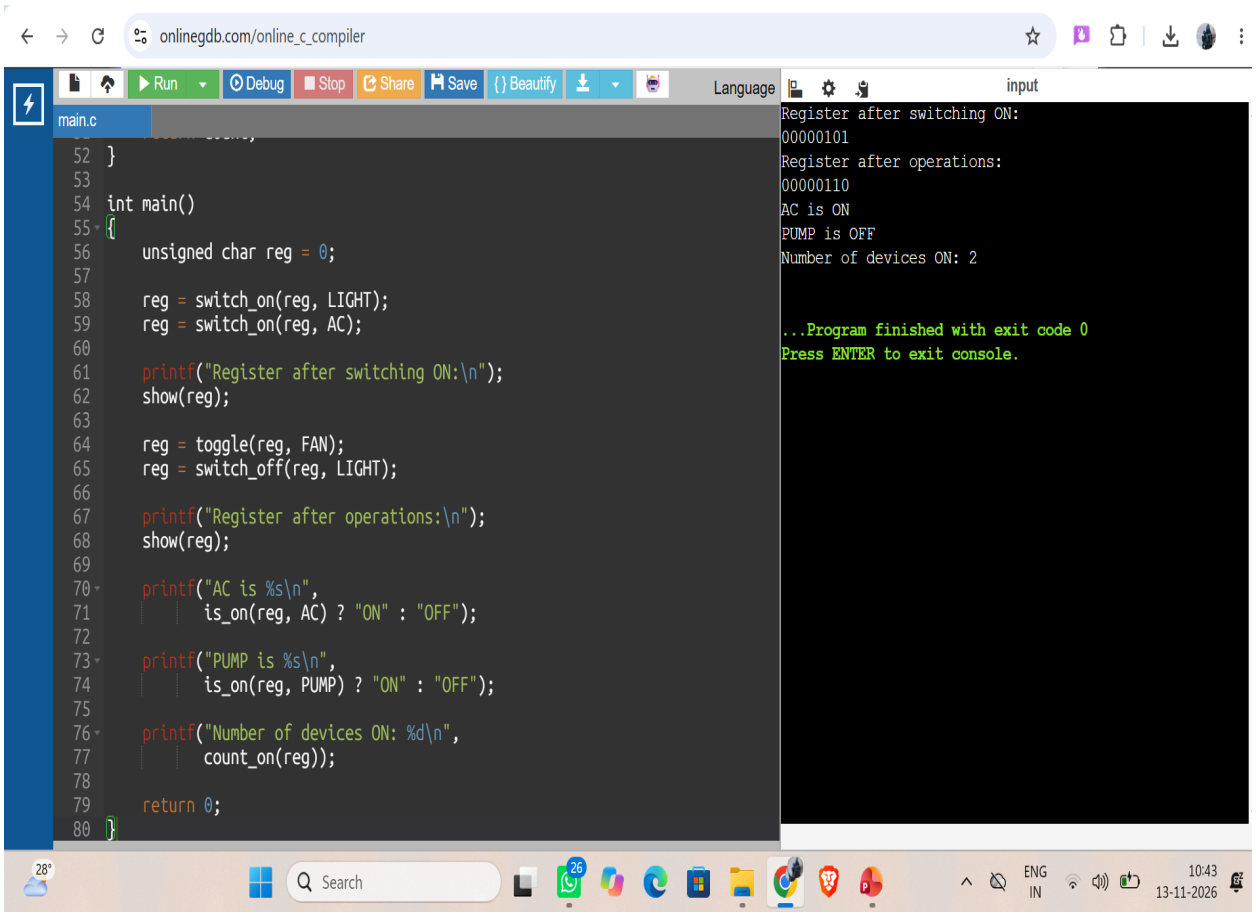
    printf("AC is %s\n",
           is_on(reg, AC) ? "ON" : "OFF");

    printf("PUMP is %s\n",
           is_on(reg, PUMP) ? "ON" : "OFF");

    printf("Number of devices ON: %d\n",
           count_on(reg));
```

```
    return 0;
}
```

3.5 output



3.6 Register calculation

Operation	Binary register	Hexadecimal
Initial	00000000	00

Light ON	00000001	01
AC ON	00000101	05
Fan toggle	00000111	07
Light OFF	00000110	06

3.7 Why are the following expressions different?

Expression 1:

```
None
reg & 1 << dev
```

Expression 2:

```
None
(reg & 1) << dev
```

In the first expression, the shift operation is performed before bitwise AND.

In the second expression, the register is first ANDed with 1 and then shifted.

Therefore, both expressions can produce different results.

3.8 Signed char problem

Using signed `char` can create problems when bit 7 is set because the value may become negative.

An unsigned character is recommended for storing binary device states.

3.9 Array alternative

An alternative is:

```
None
char state[8] = {0};


state[0] = 1;  // Light ON
state[1] = 1;  // Fan ON
state[0] = 0;  // Light OFF
```

Representation	Space per home
One unsigned char	1 byte
Eight char elements	8 bytes

For 1,000,000 homes:

- Bit-register method: approximately 1 MB.
- Array method: approximately 8 MB.

Both methods provide O(1) access to one device state. Counting ON devices requires O(8), which is O(1) for this fixed-size register.

3.10 Conclusion

- Bitwise operators provide efficient memory management.

- One byte can store eight device states.
- OR switches devices ON.
- AND clears or checks bits.
- XOR toggles device states.
- Unsigned data types are suitable for binary registers.

QUESTION 4 – ELSE-IF LADDER OR SWITCH?

4.1 Problem explanation

A university examination cell uses a C program to convert student marks into grades.

The grade classification is:

Marks	Grade
90–100	S
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	F

Outside 0–100	Invalid (?)

The program must implement grading using:

1. Else-if ladder.
2. Switch statement.

4.2 Else-if ladder explanation

An else-if ladder checks multiple conditions in sequence.

The first condition that becomes true executes its corresponding statement.

Else-if program

```
None

#include <stdio.h>

char grade_if(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';

    if (marks >= 90)
        return 'S';

    else if (marks >= 80)
        return 'A';
```

```
    else if (marks >= 70)
        return 'B';

    else if (marks >= 60)
        return 'C';

    else if (marks >= 50)
        return 'D';

    else
        return 'F';
}

int main()
{
    int marks;

    printf("Enter student marks: ");
    scanf("%d", &marks);

    printf("Grade: %c\n", grade_if(marks));

    return 0;
}
```

4.3 Switch statement explanation

A switch statement selects one case based on an expression.

The expression `marks / 10` converts the marks into a category.

For example:

- 95 / 10 = 9
- 83 / 10 = 8
- 75 / 10 = 7
- 50 / 10 = 5

The switch statement cannot directly check ranges such as 75–84. Instead, the program uses integer division to map marks to categories.

4.4 Complete C program using switch

None

```
#include <stdio.h>

char grade(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';

    switch (marks / 10)
    {
        case 10:
        case 9:
            return 'S';
```

```
        case 8:
            return 'A';

        case 7:
            return 'B';

        case 6:
            return 'C';

        case 5:
            return 'D';

        default:
            return 'F';
    }
}

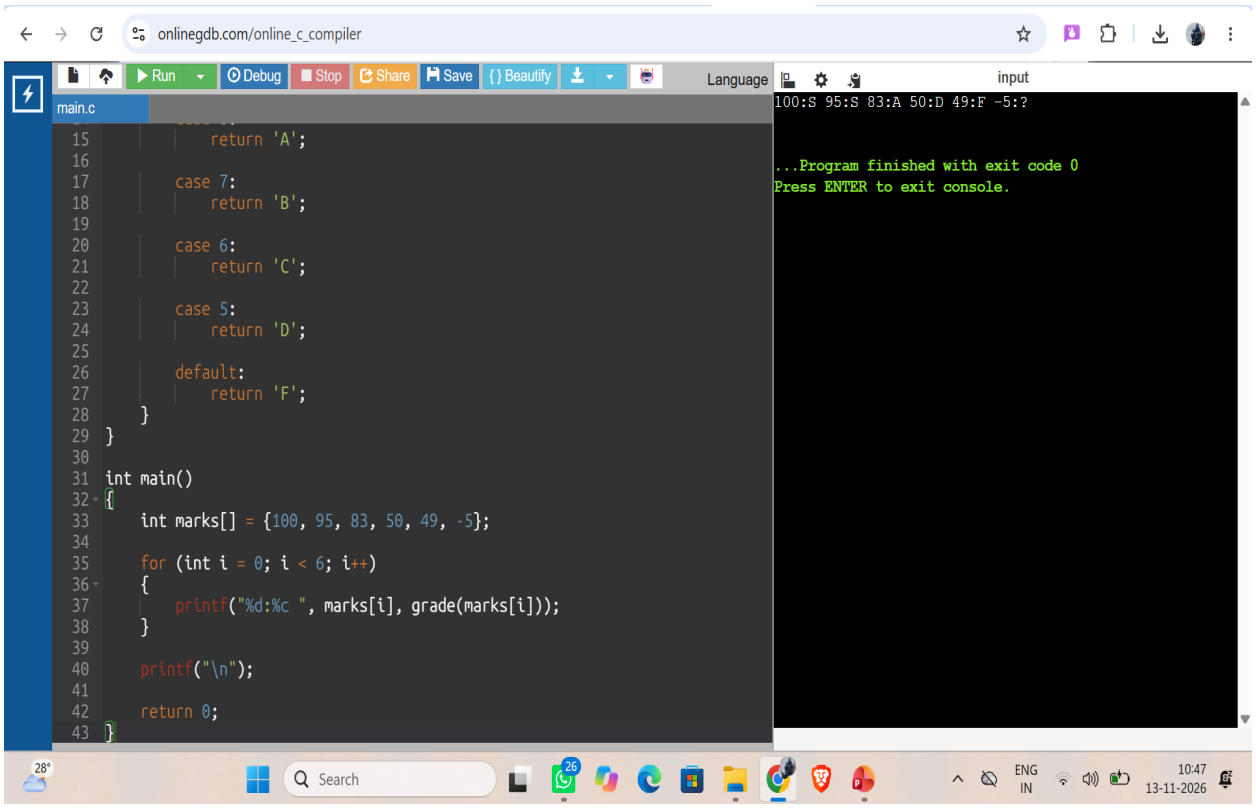
int main()
{
    int marks[] = {100, 95, 83, 50, 49, -5};

    for (int i = 0; i < 6; i++)
    {
        printf("%d:%c ", marks[i], grade(marks[i]));
    }
```

```
    printf("\n");

    return 0;
}
```

4.5 output



4.6 Effect of missing break

Consider:

```
None
switch (marks / 10)
{
```

```
    case 9:
        grade = 'S';

    case 8:
        grade = 'A';
        break;
}
```

If the student receives 95 marks, the program first assigns S. Because there is no break, it continues to the next case and changes the grade to A.

This is called fall-through.

Using `return` or `break` correctly prevents unwanted execution.

4.7 Test cases

Input marks	Expected grade	Result
100	S	Valid
90	S	Valid
89	A	Valid
80	A	Valid

75	B	Valid
60	C	Valid
50	D	Valid
49	F	Valid
0	F	Valid
-1	?	Invalid
101	?	Invalid

4.8 Scholarship classification

The scholarship decision depends on two conditions:

- Student grade.
- Family income category.

Nested if statements are suitable because both conditions must be considered.

Example:

```
None
if (grade == 'S')
{
    if (income <= 200000)
```

```
        printf("Full Scholarship");
    else
        printf("Partial Scholarship");
}
else
{
    printf("Not Eligible");
}
```

These income values are illustrative; the assignment does not specify actual scholarship income thresholds.

4.9 Comparison

Feature	Else-if	Switch
Range checking	Directly supported	Not directly supported
Multiple conditions	Yes	Limited to case values
Readability	Good for ranges	Good for fixed categories
Changing boundaries	Easy	Requires changing category mapping
Missing break	Not applicable	Can cause fall-through

4.10 Conclusion

- Else-if is suitable for checking mark ranges.
- Switch is suitable for fixed integer categories.
- Input validation prevents invalid marks.
- Missing break can produce incorrect grades.
- Nested if statements are useful for scholarship decisions.

QUESTION 5 – OPTIMIZING A PRIME-NUMBER CHECKING LOOP

5.1 Problem explanation

A railway ticket verification system uses prime-number checking to validate lucky-draw ticket numbers.

The original program checks every possible divisor from 2 to n−1.

This creates unnecessary calculations, especially when the input is a large prime number.

The objective is to reduce the number of division operations.

5.2 Prime number definition

A prime number is a positive integer greater than 1 that has exactly two factors:

1. Itself.

Examples:

- 2, 3, 5, 7, 11 and 13 are prime numbers.
- 4, 6, 8, 9 and 10 are not prime numbers.

5.3 Original prime-checking program

```
None
int is_prime(int n)
{
    if (n < 2)
        return 0;

    for (int i = 2; i < n; i++)
    {
        if (n % i == 0)
            return 0;
    }

    return 1;
}
```

The original program performs unnecessary checks because it tests divisors greater than the square root of the number.

5.4 Optimization concept

If a number has a divisor greater than its square root, it must also have a corresponding divisor smaller than its square root.

Therefore, it is sufficient to check divisors only up to √n.

For example:

For 36:

$$\sqrt{36} = 6$$

The factors are:

1, 2, 3, 4, 6, 9, 12, 18, 36.

We only need to check divisors up to 6.

5.5 Optimized C program

None

```
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)
{
    checks = 0;

    if (n < 2)
        return 0;

    for (unsigned long i = 2; i <= n / i; i++)
    {
        checks++;

        if (n % i == 0)
            return 0;
    }
```

```
    return 1;
}

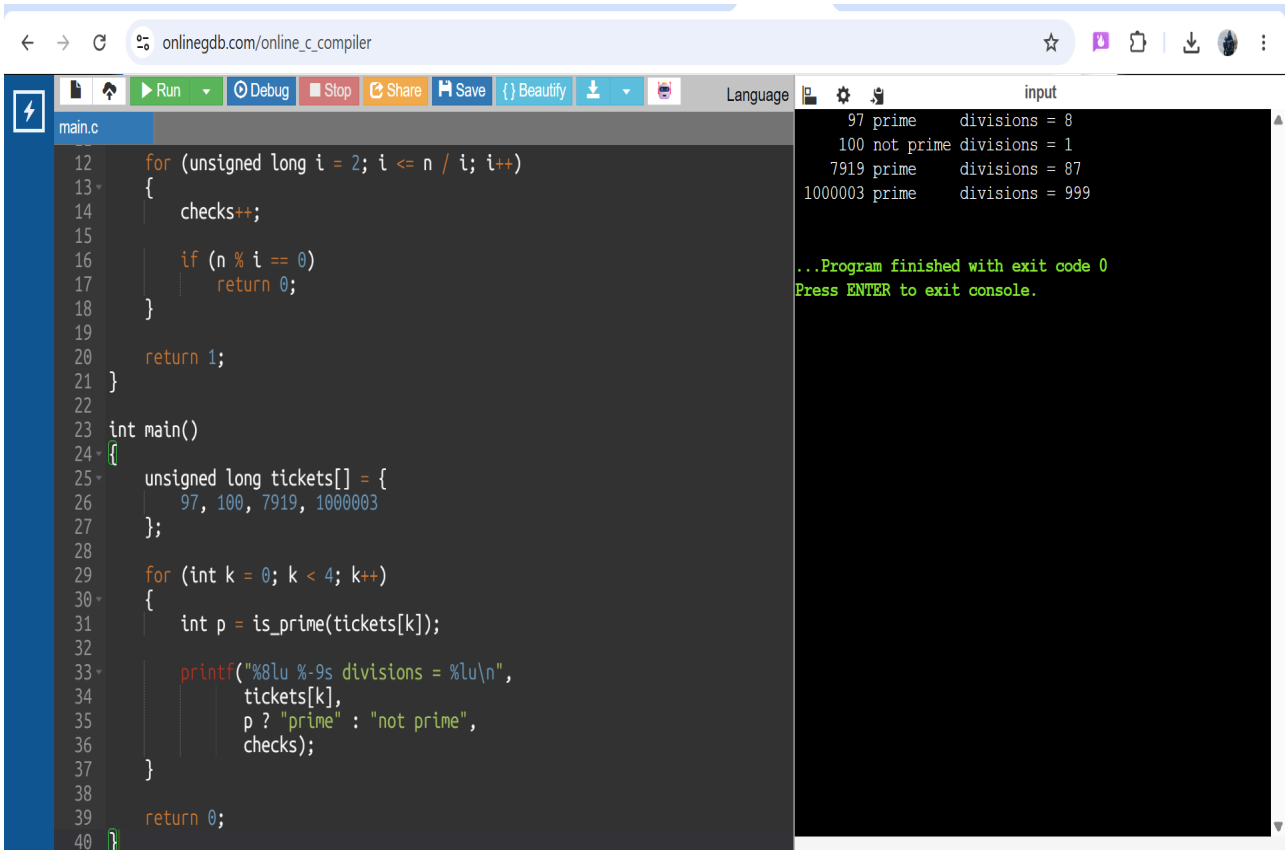
int main()
{
    unsigned long tickets[] = {
        97, 100, 7919, 1000003
    };

    for (int k = 0; k < 4; k++)
    {
        int p = is_prime(tickets[k]);

        printf("%8lu %-9s divisions = %lu\n",
               tickets[k],
               p ? "prime" : "not prime",
               checks);
    }

    return 0;
}
```

5.6 Expected output



5.7 Explanation of output

- 97 is prime. It requires 8 divisor checks.
- 100 is not prime because it is divisible by 2. It requires only 1 check.
- 7919 is prime and requires 87 checks.
- 1000003 is prime and requires 999 checks.

The number of operations is significantly reduced compared with the original loop.

5.8 Different loop versions

For loop

```
None
for (int i = 2; i <= n / i; i++)
{
```

```
    if (n % i == 0)
        return 0;
}
```

While loop

```
None
int i = 2;

while (i <= n / i)
{
    if (n % i == 0)
        return 0;

    i++;
}
```

Do-while loop

```
None
int i = 2;

if (n < 2)
    return 0;

do
{
    if (i > n / i)
```

```
        break;

    if (n % i == 0)
        return 0;

    i++;

} while (1);

return 1;
```

A do-while loop executes at least once, so the boundary must be checked carefully.

5.9 What happens if the condition is changed to i < n?

If the optimized condition is replaced with:

```
None
i < n
```

the program again checks almost every number up to n.

For a large prime, the number of division operations becomes approximately n−2.

This removes the main benefit of the optimization.

5.10 Sieve of Eratosthenes

If many ticket numbers up to a known maximum N must be checked, the Sieve of Eratosthenes can be used.

Sieve of Eratosthenes - prime numbers
algorithms-visual.com

Steps:

1. Create an array of N+1 elements.
2. Initially mark numbers as prime.
3. Mark 0 and 1 as not prime.
4. Start with 2.
5. Mark all multiples of 2 as composite.
6. Continue with the next unmarked prime.
7. Repeat until the square root of N.
8. The remaining marked numbers are prime.

5.11 Complexity comparison

Method	Time complexity	Space complexity
Original loop	O(n)	O(1)
Square-root loop	O(√n)	O(1)
Sieve preprocessing	O(N log log N)	O(N)
Sieve lookup	O(1)	Precomputed array

5.12 Conclusion

- The original program performs unnecessary divisor checks.
- The square-root method reduces the number of operations.
- The optimized program uses constant extra memory.

- The sieve is useful when checking many numbers within a fixed range.
- Choosing a suitable algorithm improves program efficiency.

QUESTION 6 – LOAN REPAYMENT SCHEDULE USING LOOPS

6.1 Problem explanation

A women's self-help group provides loans at a fixed monthly interest rate.

Each month:

1. Interest is added to the outstanding balance.
2. The member pays a fixed repayment amount.
3. The remaining balance is calculated.
4. The process continues until the loan is repaid.

The program must display the monthly repayment schedule, total interest and number of months.

6.2 Why use a while loop?

A while loop is suitable because the number of repayment months is not known in advance.

The loop continues as long as the loan balance is greater than zero.

The condition:

```
None
while (balance > 0.005)
```

uses a tolerance to avoid relying on exact floating-point equality.

6.3 Formula

$$Interest = Balance \times MonthlyRate$$

$$AmountDue = Balance + Interest$$

$$Payment = \min(Repayment, AmountDue)$$

$$ClosingBalance = AmountDue - Payment$$

6.4 Hand calculation

Given:

- Principal = Rs. 10,000
- Monthly interest = 2%
- Monthly repayment = Rs. 2,000

Amounts are rounded to two decimal places for display.

Month	Opening balance	Interest	Payment	Closing balance
1	10000.00	200.00	2000.00	8200.00
2	8200.00	164.00	2000.00	6364.00
3	6364.00	127.28	2000.00	4491.28
4	4491.28	89.83	2000.00	2581.11

5	2581.11	51.62	2000.00	632.73
6	632.73	12.65	645.38	0.00

Total interest:

$$200 + 164 + 127.28 + 89.83 + 51.62 + 12.65$$

$$= Rs.\ 645.38$$

The loan is repaid in six months.

6.5 Complete C program

```
None

#include <stdio.h>

int main()
{
    double balance, rate, repayment;
    double interest, total_interest = 0;
    double amount_due, payment;
    int month = 0;
    int limit = 360;

    printf("Enter loan amount: ");
    scanf("%lf", &balance);
```

```
printf("Enter monthly interest rate: ");
scanf("%lf", &rate);

printf("Enter monthly repayment: ");
scanf("%lf", &repayment);

if (balance <= 0 || rate < 0 || repayment <= 0)
{
    printf("Invalid loan details\n");
    return 1;
}

rate = rate / 100.0;

printf("\nMONTHLY REPAYMENT SCHEDULE\n");

while (balance > 0.005 && month < limit)
{
    interest = balance * rate;

    amount_due = balance + interest;

    if (repayment < amount_due)
        payment = repayment;
    else
```

```
            payment = amount_due;

        double closing = amount_due - payment;

        total_interest += interest;

        month++;

        printf("\nMonth: %d\n", month);
        printf("Opening Balance: %.2f\n", balance);
        printf("Interest: %.2f\n", interest);
        printf("Payment: %.2f\n", payment);
        printf("Closing Balance: %.2f\n", closing);

        balance = closing;

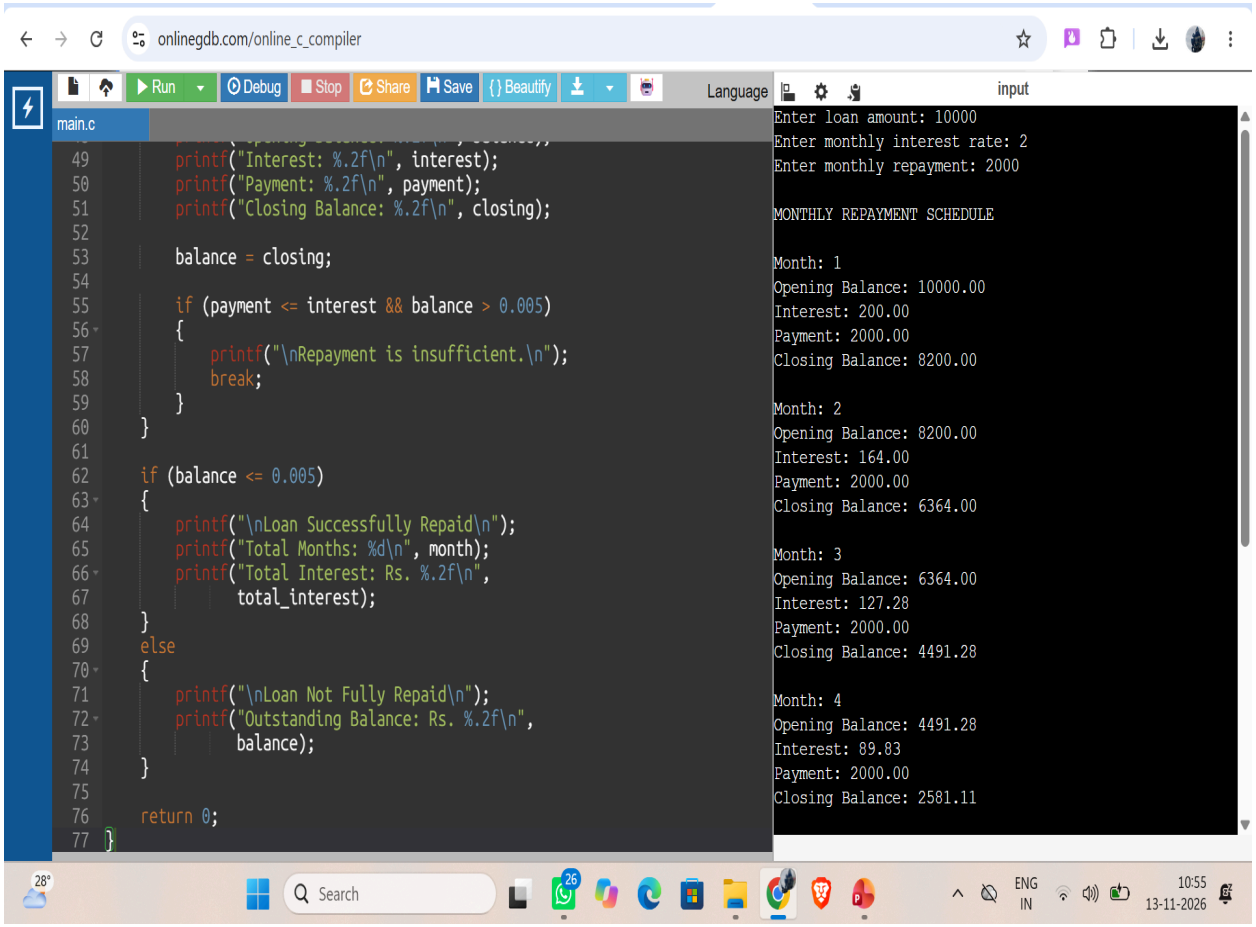
        if (payment <= interest && balance > 0.005)
        {
            printf("\nRepayment is insufficient.\n");
            break;
        }
    }

    if (balance <= 0.005)
    {
        printf("\nLoan Successfully Repaid\n");
```

```
        printf("Total Months: %d\n", month);
        printf("Total Interest: Rs. %.2f\n",
               total_interest);
    }
    else
    {
        printf("\nLoan Not Fully Repaid\n");
        printf("Outstanding Balance: Rs. %.2f\n",
               balance);
    }

    return 0;
}
```

6.6 output



6.7 What happens if repayment is smaller than monthly interest?

For example:

- Balance = Rs. 10,000
- Interest = 2%
- Interest amount = Rs. 200
- Repayment = Rs. 150

The payment is smaller than the interest.

Therefore, the outstanding loan increases instead of decreasing.

The program should detect this condition and report that the repayment is insufficient.

6.8 Safety limit

The program uses:

```
None
int limit = 360;
```

This prevents the loop from running indefinitely.

If the limit is reached, the program displays the outstanding loan balance instead of incorrectly reporting that the loan is repaid.

6.9 Formula alternative

The assignment provides the formula:

$$n = \frac{-\log(1-rP/E)}{\log(1+r)}$$

Where:

- P = Principal.
- r = Monthly interest rate.
- E = Monthly repayment.
- n = Number of months.

The formula estimates the repayment duration under its assumptions.

Method	Time	Space
Monthly simulation	O(n)	O(1)
Direct formula	O(1)	O(1)

The formula is faster for estimating the number of months, while simulation gives a detailed repayment schedule.

6.10 Conclusion

- A while loop is suitable for loan repayment.
- Monthly interest must be calculated before payment.
- The final payment must not exceed the remaining amount due.
- Insufficient repayment must be detected.
- A safety limit prevents uncontrolled loops.
- Floating-point precision must be considered.

QUESTION 7 – DIAGNOSE, REDESIGN AND VALIDATE

7.1 Problem explanation

This question requires selecting one program from Questions 1–6 that has a significant limitation when the input size or requirements change.

The selected program is:

Prime Number Checking Program

The original program checks every divisor from 2 to n−1. This causes unnecessary calculations when checking large prime numbers.

The redesigned program uses the mathematical property that a composite number must have a divisor less than or equal to its square root.

7.2 A. Diagnose the bottleneck

The original code is:

```
None
for (int i = 2; i < n; i++)
{
    if (n % i == 0)
        return 0;
}
```

The main bottleneck is the repeated modulo operation:

```
None
n % i
```

For a large prime number, the loop performs almost n division checks.

For example, when n = 1,000,003, the original algorithm performs 1,000,001 checks.

This makes the original program inefficient for large input values.

7.3 B. Define the design objective

The objective is to reduce the number of division operations while maintaining correct prime-number detection.

The redesign should:

1. Reduce unnecessary divisor checks.
2. Improve execution speed.
3. Handle large prime numbers.
4. Maintain correct output.
5. Use minimum additional memory.

Measurable objective: Reduce the number of divisor checks for 1,000,003 from approximately one million to 999.

7.4 C. Design the revised solution

The improved algorithm uses the square-root property.

Changes made:

- Check whether the number is less than 2.
- Check whether the number is divisible by 2.
- Skip all even divisors after that.
- Check only odd divisors.
- Stop when the divisor exceeds the square root.

Revised pseudocode

```
None
START

Read number n

IF n < 2
    Return NOT PRIME

IF n = 2
    Return PRIME

IF n is divisible by 2
    Return NOT PRIME

Set i = 3
```

```
WHILE i <= n / i

    IF n is divisible by i
        Return NOT PRIME

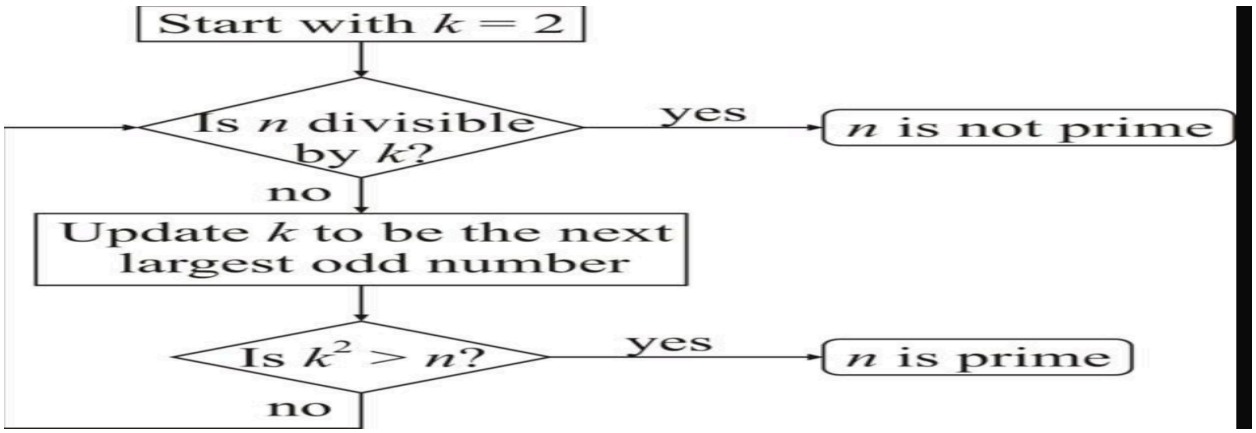
    Increase i by 2

END WHILE

Return PRIME

STOP
```

7.5 Flowchart



7.6 Revised C program

None

```c
#include <stdio.h>

int is_prime_optimized(unsigned long n,
                       unsigned long *checks)
{
    *checks = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    (*checks)++;

    if (n % 2 == 0)
        return 0;

    for (unsigned long i = 3;
         i <= n / i;
         i += 2)
    {
        (*checks)++;

        if (n % i == 0)
            return 0;
```

```
    }

    return 1;
}

int main()
{
    unsigned long numbers[] = {
        97, 100, 7919, 1000003
    };

    unsigned long checks;
    int result;

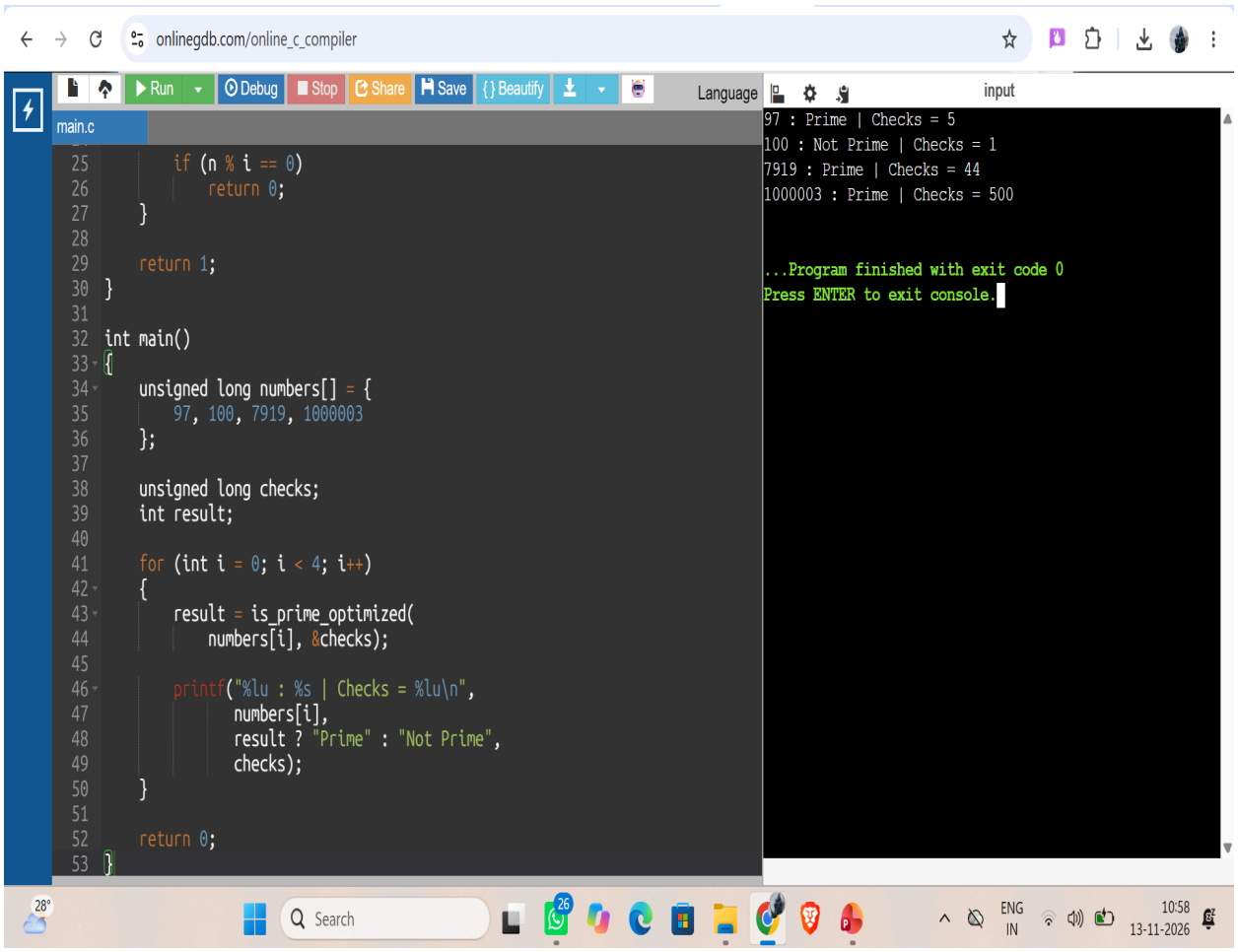
    for (int i = 0; i < 4; i++)
    {
        result = is_prime_optimized(
            numbers[i], &checks);

        printf("%lu : %s | Checks = %lu\n",
               numbers[i],
               result ? "Prime" : "Not Prime",
               checks);
    }

    return 0;
```

```
}
```

OUTPUT:



7.7 D. Validate the redesigned solution

Test the program using different inputs.

Input	Expected result	Reason

-3	Not Prime	Negative number
0	Not Prime	Less than 2
1	Not Prime	Less than 2
2	Prime	Smallest prime
3	Prime	Prime number
4	Not Prime	Divisible by 2
9	Not Prime	Divisible by 3
97	Prime	Prime number
100	Not Prime	Composite number
1000003	Prime	Large prime

For the large prime, the square-root loop checks odd divisors from 3 to 999, because it skips even divisors.

The optimized version therefore performs fewer modulo checks than the original.

7.8 E. Compare the original and redesigned programs

Feature	Original	Redesigned
Divisor range	2 to n−1	Up to √n
Even divisors	All checked	Skipped after 2
Time complexity	O(n)	O(√n)
Extra space	O(1)	O(1)
Large input performance	Slow	Faster
Correctness	Correct	Correct

7.9 Trade-offs

Advantages:

- Reduces unnecessary calculations.
- Improves performance for large prime numbers.
- Does not require additional arrays.
- Maintains constant auxiliary memory.

Limitations:

- Still performs multiple divisor checks for large prime numbers.
- If millions of numbers within a fixed range must be tested, the Sieve of Eratosthenes may be more efficient overall.
- The redesigned code requires careful handling of boundary conditions.

7.10 Conclusion

The redesigned prime-number algorithm improves performance by using mathematical properties rather than checking every possible divisor.

The optimized solution reduces time complexity from O(n) to O(√n), while maintaining O(1) auxiliary space. It demonstrates how identifying a bottleneck and selecting an appropriate algorithm can improve program efficiency.